**EEE4120F**

## High Performance Embedded Systems

# REFERENCE GUIDE

OPEN MP Basics

### Course Team


**Course Convener:** Associate Professor Simon Winberg

**Teaching Assistant:** Travimadox Webb

# OpenMP: Multi-Processing

A Short Reference Guide

## 1 Introduction

OpenMP (Open specifications for Multi Processing) is an API that supports multi-threaded, shared address space parallelism. It is defined by major hardware and software vendors to ensure portability while allowing for hardware-specific optimizations.

**Tip**

**Learning Resources:**

- Oxford Research Software Engineering Tutorial
- Official OpenMP Website

## 2 Verifying OpenMP Support

OpenMP is a compiler feature, not a standalone program. To verify it works, you must check if your compiler supports the OpenMP flag.

### 2.1 Linux (Ubuntu/Debian)

Most Linux distributions come with GCC pre-installed, which supports OpenMP out of the box.

To check if OpenMP is in your gcc run:

```
echo | gcc -E -dM -fopenmp - | grep -i openmp
```

If OpenMP is present it will show

```
#define _OPENMP 201511
```

**Warning**

**Common Error:** `fatal error: omp.h: No such file or directory`
**Fix:** You are missing the development library. Run: `sudo apt install libomp-dev`

### 2.2 Windows

If you dont have MinGW installed check MinGW

Run the follwing command.

```
gcc -fopenmp -dM -E - < NUL | findstr "OPENMP"
```

If OpenMP is present it will show

```
#define _OPENMP 201511
```

# 3 Compiling OpenMP Programs

Once OpenMP support is verified, compiling your code requires telling the compiler to enable the parallel directives.

## 3.1 Linux (and Windows MinGW)

Since OpenMP is a compiler flag, the command is identical for GCC on Linux and MinGW on Windows.

1. **Basic Compilation:** Add the `-fopenmp` flag.

   ```
   gcc -fopenmp my_program.c -o my_program
   ```

2. **With Optimization:** It is common to add optimization flags for parallel code.

   ```
   gcc -O2 -fopenmp my_program.c -o my_program
   ```

**Warning**

**Common Mistake:** If you forget `-fopenmp`, the code will compile without errors, but it will run **sequentially** (on a single thread) because the compiler ignores the `#pragma` directives.

# 4 OpenMP Basics

OpenMP uses the **Fork-Join** model of parallel execution:

1. **Master Thread:** All programs begin as a single process (master thread) executing sequentially.
2. **Fork:** When a parallel region construct is encountered, the master thread creates a team of parallel threads.
3. **Parallel Execution:** Statements inside the construct are executed in parallel by the team.
4. **Join:** Threads synchronize and terminate at the end of the region, leaving only the master thread to continue.

The API consists of three primary components:

- **Compiler Directives:** Imbedded in C source code to specify parallelism.
- **Runtime Library Routines:** Access runtime parameters (e.g., thread counts).
- **Environment Variables:** Control runtime behavior.

# 5 Compiler Directives

Directives follow C conventions (case-sensitive). Long lines can be continued with a backslash (\).

## 5.1 The Parallel Construct

This is the fundamental construct that defines a parallel region. Code within this block is duplicated and executed by all threads.

```
#pragma omp parallel [clause...]
{
    // Structured block executed by all threads
}
```

**Warning**

**Implicit Barrier:** There is an implied barrier at the end of a parallel section. Only the master thread continues execution past this point.

## 5.2 Work-Sharing Constructs

Work-sharing constructs divide the execution of code among the team members. They do *not* launch new threads.

### 5.2.1 The For Directive

Used to split loop iterations among threads.

```
#pragma omp for [clause...]
for (i = 0; i < N; i++) {
    // Loop body
}
```

**Scheduling Clauses:**

- `schedule(static [, chunk])`: Iterations are divided into fixed pieces. If chunk is unspecified, they are divided evenly.
- `schedule(dynamic [, chunk])`: Iterations are assigned dynamically as threads finish their previous work. Default chunk size is 1.

## 5.3 Synchronization Directives

Coordinate work between multiple threads.

- **master:** Region executed *only* by the master thread. Other threads skip it.
- **critical:** Region executed by only one thread at a time.
- **barrier:** Synchronizes all threads; they wait until everyone reaches this point.
- **atomic:** Ensures a specific memory location is updated atomically (miniature critical section).
- **ordered:** Ensures loop iterations execute in serial order.

### 5.4 Data Scope Attributes

Since OpenMP uses a shared address space, understanding scope is vital.

- **Shared:** Global variables visible before the parallel region are shared by default.
- **Private:** Variables local to a thread (e.g., loop indices, stack variables).

You can explicitly control scoping with clauses: `private`, `shared`, `firstprivate`, `reduction`, etc.

## 6 Runtime Library Routines

Include `omp.h` to access these functions.

### 6.1 Thread Management

```
// Set number of threads for next parallel region
void omp_set_num_threads(int num_threads);

// Get number of threads in current team
int omp_get_num_threads();

// Get ID of the current thread (0 to N-1)
int omp_get_thread_num();

// Get max threads possible
int omp_get_max_threads();

// Get total number of processors available
int omp_get_num_procs();
```

### 6.2 Locks

General purpose locking routines (semaphores).

```
// Initialize lock (Initial state: unlocked)
void omp_init_lock(omp_lock_t *lock);

// Destroy lock
void omp_destroy_lock(omp_lock_t *lock);

// Wait until lock is available, then take it
void omp_set_lock(omp_lock_t *lock);

// Release the lock
void omp_unset_lock(omp_lock_t *lock);

// Attempt to set lock (non-blocking)
// Returns true if successful
int omp_test_lock(omp_lock_t *lock);
```

**Warning**

**Illegal Operation:** It is illegal to call `omp_destroy_lock` with a lock variable that is not initialized.

## 7 Environment Variables

You can control execution behavior using environment variables in your shell.

```
# Set maximum number of threads
setenv OMP_NUM_THREADS 8
```

**Tip**

**Precedence:** The `omp_set_num_threads` function call in your code overrides the `OMP_NUM_THREADS` environment variable.